

集合框架

1. 集合框架概述

- 含义

集合框架是为表示和操作集合而规定的一种统一的、标准的体系结构。

- 集合框架的三大组成部分

- 接口：表示集合的抽象数据类型
- 实现：接口的具体实现，即一系列数据结构
- 算法：含有查找、排序等算法的工具包

说明

- 所有集合类都位于 `java.util` 包下。
- 集合框架可以细化为四个部分
 - 数据结构：列表 `List`、队列 `Queue`、双端队列 `Deque`、集合 `Set` 和映射 `Map`
 - 比较器：比较器 `Comparator` 和排序接口 `Comparable`
 - 算法：常用算法类 `Collections` 和静态数组的排序、查找类 `Arrays`
 - 迭代器：通用迭代器 `Iterator` 和 `List` 特化迭代器 `ListIterator`

- 集合类只容纳对象。

2. Collection 接口

- `Collection` 接口是 `List`、`Set` 和 `Queue` 接口的父接口，是集合框架的根接口。

```
1 // Collection 接口中定义的常用函数
2 public interface Collection<E> extends Iterable<E> {
3     boolean add(E e);
4     boolean addAll(Collection<? extends E> c);
5     void clear();
6     boolean remove(E e);
7     boolean removeAll(Collection<?> c);
8     boolean contains(Object obj);
9     boolean containsAll(Collection<?> c);
10    boolean isEmpty();
11    int size();
12    Object[] toArray();
13    Iterator iterator();
14    // ...
15 }
```

- `List` 接口

- `List` 的特征是其元素有确定的顺序并且可以重复，可以通过下标访问元素。

```

1 // List 接口中新增的函数
2 public interface List<E> extends Collection<E> {
3     E get(int index);
4     E set(int index, E element);
5     void add(int index, E element);
6     E remove(int index);
7     int indexOf(Object o);
8     int lastIndexOf(Object o);
9     boolean addAll(int index, Collection<? extends E> c);
10    ListIterator<E> listIterator();
11    ListIterator<E> listIterator(int index);
12    List<E> subList(int fromIndex, int toIndex);
13    // ...
14 }

```

◦ List 的实现类有 ArrayList 和 LinkedList。

- ArrayList 实现了在内存中连续储存的可变长数组，遍历和随机访问元素的效率较高。
- LinkedList 实现了链表，插入和删除的效率较高。

```

1 // LinkedList 的特殊函数
2 {
3     void addFirst(E element);
4     void addLast(E element);
5     E getFirst();
6     E getLast();
7     E removeFirst();
8     E removeLast();
9 }

```

• Set 接口

- Set 中的元素必须唯一。
- 添加到 Set 中的元素必须定义 equals 函数，从而保证元素的唯一性。
- 实现 Set 接口的类有 HashSet、TreeSet 等。

3. Map 接口

- Map 接口专门处理键值对映射数据的存储，其常用的成员有

```

1 public interface Map<K, V> {
2     int size();
3     boolean isEmpty();
4     boolean containsKey(Object key);
5     boolean containsValue(Object value);
6     V get(Object key);
7     V getOrDefault(Object key, V defaultValue);
8     // 若找到 key 返回其对应的 value, 否则返回 defaultVaule
9     V put(K key, V value);
10    V remove(Object key);

```

```

11     void putAll(Map<? extends K, ? extends V> m);
12     void clear();
13     Set<K> keySet();
14     Collection<V> values();
15     Set<Map.Entry<K, V>> entrySet();
16
17     interface Entry<K, V> {
18         K getKey();
19         V getValue();
20         V setValue();
21         // ...
22     }
23
24     //...
25 }

```

Map 的注意事项

- Map 是接口，不能直接实例化。通常选用其子类 TreeMap 或 HashMap。

Map 实现类	TreeMap	HashMap
底层结构	红黑树	哈希桶
增删查时间复杂度	$O(\log_2 N)$	$O(1)$
是否有序	关于 key 有序	无序
线程安全性	不安全	不安全
增删查区别	需要进行元素比较	通过哈希函数计算哈希地址
注意事项	key 必须能够比较，否则抛 ClassCastException	自定义类型必须重写 equals 和 hashCode 函数
应用场景	需要 key 有序的情形	关心更高的时间性能

- Map 中的 key 是唯一的，value 可以重复。分离 key 时，将其储存到 Set 中；分离 value 时，将其储存到 Collection 的可包含重复元素的子类中。
- Map 中的 key 不能修改，value 可以修改。如果想修改 key 只能先删除后重新插入。
- Map.Entry —— Map 中的每一个元素，即将键值对打包后的类型

```

1 // 使用 Map.Entry 遍历 Map
2 import java.util.*;
3
4 public class Test {
5     public static void main(String[] args) {
6         Map<Integer, String> map = new TreeMap<Integer, String>();
7         map.put(10, "hello"); map.put(20, "abc"); map.put(66,
            "yes");

```

```

8
9     for (Map.Entry<Integer, String> entry : map.entrySet()) {
10         System.out.println("key: " + entry.getKey()
11                             + " value: " + entry.getValue());
12     }
13 }
14 }

```

- 另一种遍历的有效手段

```

1 // 使用 forEach 遍历 Map
2 import java.util.*;
3
4 public class Test {
5     public static void main(String[] args) {
6         Map<Integer, String> map = new TreeMap<Integer, String>();
7         map.put(10, "hello"); map.put(20, "abc"); map.put(66,
8         "yes");
9
10        map.forEach((key, value) -> {
11            System.out.println("key: " + key + " value: " + value);
12        })
13    }
14 }

```

- 使用 `TreeMap` 时, 如果 `key` 是自定义类型, 必须定义比较大小的方法。

- Method 1

```

1 // Test1.java
2 import java.util.*;
3
4 class Person implements Comparable {
5     private String name;
6     private int age;
7     public Person(String name, int age) { this.name = name; this.age =
8     age; }
9
10    @Override
11    public int compareTo(Object o) {
12        Person p = (Person)o;
13        if (this.age == p.age) return this.name.compareTo(p.name);
14        else return (this.age < p.age) ? -1 : 1;
15    }
16 }
17
18 public class Test1 {
19     public static void main(String[] args) {
20         Map<Person, String> map = new TreeMap<Person, String>();
21         map.put(new Person("Tom", 20), "hello");
22         map.put(new Person("Jana", 18), "world");
23     }
24 }

```

```
22 }
```

◦ Method 2

```
1 // Test2.java
2 import java.util.*;
3
4 class Person implements Comparable {
5     private String name;
6     private int age;
7     public Person(String name, int age) { this.name = name; this.age =
age; }
8 }
9 public class Test2 {
10     public static void main(String[] args) {
11         Map<Person, String> map = new TreeMap<Person, String>
12             (new Comparator<Person>() {
13                 @Override
14                 public int compare(Person p1, Person p2) {
15                     if (p1.age == p2.age) return
p1.name.compareTo(p2.name);
16                     else return (p1.age < p2.age) ? -1 : 1;
17                 }
18             });
19         map.put(new Person("Tom", 20), "hello");
20         map.put(new Person("Jana", 18), "world");
21     }
22 }
```

• 更多的细节

- Vector 和 ArrayList 实现原理相同、功能相同，多数情况可以互用。区别为(1) Vector 线程安全，ArrayList 重速度、线程不安全；(2)长度需增加时，Vector 默认增长一倍，ArrayList 默认增长50%。
- Hashtable 和 HashMap 实现原理相同、功能相同，多数情况可以互用。区别为(1) Hashtable 继承自 Dictionary 类，HashMap 实现 Map 接口；(2) Hashtable 线程安全，HashMap 线程不安全；(3) Hashtable 不允许 null 值。
- 关于 null
 - 各集合中均可以放 null 值。
 - Set 的元素和 Map 的 key 中，null 最多有一个。特别，TreeMap 的 key 不能是 null。

4. Iterator 接口

```
1 public interface Iterator<E> {
2     boolean hasNext();
3     E next();
4     void remove();
5 }
```

```
1 // 遍历一个 List 的四种方式
2 import java.util.*;
3
4 public class TraverseList<E> {
5     public void traverseByFor(List<E> list) {
6         for (int i = 0; i < list.size(); i++)
7             System.out.println(list.get(i));
8     }
9     public void traverseByPoweredFor(List<E> list) {
10        for (E each : list)
11            System.out.println(each);
12    }
13    public void traverseByIterator(List<E> list) {
14        Iterator<E> it = list.iterator();
15        while (it.hasNext())
16            System.out.println(it.next());
17    }
18    public void traverseByForEach(List<E> list) {
19        list.forEach((e) -> {
20            System.out.println(e);
21        });
22    }
23 }
```

异常 (Exception)

1. 异常及其处理

- 异常是一个可以正确运行的程序在运行中可能发生的错误；异常处理就是将错误按类型和差别区分，对无法预知的错误进行捕获，并为之编写错误处理代码。
- 异常的特点
 - 偶然性
 - 可预见性
 - 严重性
- Java 异常类
 - `Throwable` 是 `Object` 的直接子类，是所有异常类的父类，它位于 `java.lang` 包中。
 - `Exception` 继承自 `Throwable`，它本身及其子类都是异常。
 - `Error` 也是 `Throwable` 的子类，它本身及其子类都由 Java 虚拟机生成并抛出，程序不做处理。
- 异常的分类
 - **受检异常**：非 `RuntimeException` 异常和用户自己定义的未继承自 `RuntimeException` 的异常。此类异常必须用 `try-catch` 处理，或者该函数 `throws` 该异常。

```
1 java.lang.ClassNotFoundException
2 java.lang.CloneNotSupportedException
3 java.lang.IllegalAccessException
4 java.lang.NoSuchMethodException
5 java.io.IOException
6 java.io.FileNotFoundException
7 ...
```

- **非受检异常**：由系统检测，用户程序可不做处理。

```
1 java.lang.ArithmeticException
2 java.lang.ArrayStoreException
3 java.lang.ClassCastException
4 java.lang.IllegalArgumentException
5 java.lang.NumberFormatException
6 java.lang.IndexOutOfBoundsException
7 java.lang.ArrayIndexOutOfBoundsException
8 java.lang.StringIndexOutOfBoundsException
9 java.lang.NullPointerException
10 ...
```

错误 `Error` 也是非受检的。

```
1 java.lang.LinkageError
2 java.lang.NoClassDefFoundError
3 java.lang.OutOfMemoryError
4 java.lang.StackOverflowError
5 java.lang.UnknownError
6 java.lang.VirtualMachineError
7 ...
```

2. 异常处理的抓抛模型

- 自定义异常 `Exceptions.java`

```
1 class ExpA extends Exception {      // 自定义异常可以继承自 Exception
2
3 }
4 class ExpB extends Throwable {      // 也可以继承自 Throwable
5     // 对于继承自 Throwable 的异常, 不能用 Exception 捕获。
6     // 因为 Exception 是 Throwable 的子类。
7 }
8 class ExpC extends RuntimeException {
9     // 自定义的非受检异常
10 }
```

- 编写可能抛出异常的类 `MyClass.java`

```
1 import java.io.IOException;
2 import java.util.Random;
3
4 public class MyClass {
5     void throwMyException() throws ExpA { // 声明函数抛出异常
6         throw new ExpA();
7     }
8     void throwMoreExceptions() throws ExpB, IOException, Exception {
9         // 声明函数抛出一系列异常
10
11         Random r = new Random();
12         int x = r.nextInt(-1, 2); // return {-1, 0, 1} randomly
13         if (x > 0)
14             throw new ExpB();
15         else if (x == 0)
16             throw new IOException();
17         else
18             throw new Exception();
19     }
20     void throwUncheckedException(int idx) {
21         int[] arr = new int[10];
22         arr[idx] = 10;
23     }
24     void throwSelfDefUncheckedException() { // 运行时异常不声明也不报错
25         throw new ExpC();
26     }
27 }
```


- 编写测试类，用 try-catch(-finally) 处理异常 Test.java

```
1  import java.io.IOException;
2
3  public class Test {
4      public static void main(String[] args) {
5          MyClass mc = new MyClass();
6
7          try {
8              mc.throwMyException();
9          } catch (ExpA e) {
10             e.printStackTrace(System.out);
11             System.out.println("catch ExpA");
12         }
13
14         try {
15             mc.throwMoreExceptions();
16         } catch (ExpB e) {
17             e.printStackTrace(System.out);
18             System.out.println("catch ExpB");
19         } catch (IOException e) {
20             e.printStackTrace(System.out);
21             System.out.println("catch IOException");
22         } catch (Exception e) {
23             // 更上层的异常类应放 catch 的后面，因为父类异常可以捕获子类异常
24             e.printStackTrace(System.out);
25             System.out.println("catch Exception");
26         }
27
28         try {
29             mc.throwUncheckedException(3);
30         } catch (Exception e) {
31             // 用父类 Exception 捕获子类 ArrayIndexOutOfBoundsException
32             System.out.println("in catch");
33         } finally {
34             // 无论是否抛出并捕获，finally 都会执行
35             System.out.println("in finally");
36         }
37
38         try {
39             mc.throwUncheckedException(100);
40         } catch (Throwable t) {
41             // 用父类 Throwable 捕获子类 ArrayIndexOutOfBoundsException
42             System.out.println("in catch");
43         } finally {
44             // 无论是否抛出并捕获，finally 都会执行
45             System.out.println("in finally");
46         }
47
48         mc.throwSelfDefUncheckedException(); // 非受检异常，编译不报错
```

```
49     }
50 }
```

• 注意事项

- 一个函数可以不处理它产生的异常，而是通过 throws 关键字**声明异常**，从而使异常**沿着调用层次向上传递**，由调用它的函数处理。当传递到主函数 main 而没有处理异常，即 `public static void main(String[] args) throws Exception { }`，则**异常将交给系统处理**。
- **子类重写父类的函数时，不能抛出比父类函数抛出范围更大的异常。**

说明

使用动多态时，通过父类的引用动态调用子类重写的函数。在调用处，由于使用了父类引用，故只会用 try-catch 处理父类抛出的异常类型，而**无法预知**子类可能产生的更多的异常类型，则会出错。

- finally 细节：try 中抛出异常 - catch 中 return - finally 中修改返回值
 - 此类情况下，程序会**自动添加一个返回值类型的、名为var2的变量**。在 catch 中的 return x; 语句会被换成 var2 = x;。真正的 return 发生在 finally 执行之后，并且是 return var2;。

```
1 public class Test {
2     public static void main(String[] args) {
3         System.out.println("test1: in main, i = " + test1());
4         System.out.println();
5         System.out.println("test2: in main, s = " + test2());
6         System.out.println();
7         System.out.println("test3: in main, sb = " + test3());
8     }
9     public static int test1() {
10        int i = 1;
11        // int var2;
12        try {
13            throw new Exception();
14        } catch (Exception e) {
15            return i;                // var2 = i; (var2 == i == 1)
16        } finally {
17            i++;                    // i == 2;
18            System.out.println("test1: in finally, i = " + i);
19        }
20        // return var2;
21    }
22    public static String test2() {
23        String s = "Hello";
24        // String var2;
25        try {
26            throw new Exception();
27        } catch (Exception e) {
28            return s;                // var2 = s;
29        } finally {                // 字符串的 + 会 new 出一个新字符串
30            s += " World";          // var2 -> "Hello"; s -> "Hello
World"
```

```

31         System.out.println("test2: in finally, s = " + s);
32     }
33     // return var2;
34 }
35 public static StringBuilder test3() {
36     StringBuilder sb = new StringBuilder("Good");
37     // StringBuilder var2;
38     try {
39         throw new Exception();
40     } catch (Exception e) {
41         return sb;           // var2 = s;
42     } finally {
43         sb.append(" Morning"); // 不 new 新对象
44                                // sb -> "Good Morning" <- var2
45         System.out.println("test3: in finally, sb = " + sb);
46     }
47     // return var2;
48 }
49 }

```

```

1 // 输出
2 test1: in finally, i = 2
3 test1: in main, i = 1
4
5 test2: in finally, s = Hello World
6 test2: in main, s = Hello
7
8 test3: in finally, sb = Good Morning
9 test3: in main, sb = Good Morning

```

Java IO

(一) 基本输入输出

1. 命令行参数

```
1 public class CommandLineArgsTest {
2     public static void main(String[] args) {
3         for (String arg : args)
4             System.out.println(arg);
5     }
6 }
```

```
1 >>> javac CommandLineArgsTest.java
2 >>> java CommandLineArgsTest arg0 arg1 arg2 ...
3 arg0
4 arg1
5 arg2
6 ...
7 >>>
```

`args[0]` 从第一个有效参数开始，不包括程序名称。

2. 标准输入输出

- 三个标准 I/O 对象
 - 标准输入 `System.in`
 - 标准输出 `System.out`
 - 标准错误 `System.err`
- 两个标准输出对象都重载有 `println` 和 `print` 函数，支持基本类型和类类型的输出。打印类类型时，调用其 `toString` 函数。
- `java.util.Scanner` 的使用
 - `Scanner` 通过分隔符模式将输入分解为标记（默认情况下为空白符匹配），然后可以使用不同的 `next` 函数获取其值。在获取前，一般需要判断是否还有要输入的数据。

```
1 import java.util.*;
2
3 public class Test {
4     static Scanner in = new Scanner(System.in);
5     public static ArrayList<String> readWord() {
6         ArrayList<String> ret = new ArrayList<String>();
7         while (in.hasNext()) {    // 判断是否还有数据
8             ret.add(in.next());    // 读取数据
9         }
10        return ret;
11    }
```

```

11     }
12     public static ArrayList<String> readLine() {
13         ArrayList<String> ret = new ArrayList<String>();
14         while (in.hasNextLine()) {
15             ret.add(in.nextLine());
16         }
17         return ret;
18     }
19     public static void readAndShow() {
20         int i = 0;
21         double d = 0;
22         String s = null;
23         if (in.hasNextInt()) i = in.nextInt();
24         if (in.hasNextDouble()) d = in.nextDouble();
25         if (in.hasNext()) s = in.next();
26         System.out.println(String.format("i = %d, d = %.2f, s = %s", i, d,
27             s));
28     }

```

(二) 文件与数据流

1. 输入输出流

- **概念：一组有顺序的、有起点和终点的字节集合。**
 - Java 程序不能直接操纵 I/O 设备，而是在程序和设备之间加入一个中介介质，**建立起数据通道传输数据**，这就是流。
 - **流是数据传输的抽象表达，和具体设备无关。**程序一旦建立起流，就可以不用关心起点或终点是何种设备。
- **分类**
 - **输入数据流：只读不写**
 - 字节流：InputStream 的子类
 - 字符流：Reader 的子类
 - **输出数据流：只写不读**
 - 字节流：OutputStream 的子类
 - 字符流：Writer 的子类

2. 文件类 File

- File 类代表与平台无关的文件或目录。
- File 类不涉及文件的具体内容，一般用于获得文件属性、新建和删除、重命名等。
- File 类常用操作

```

1 // 四种构造

```

```

2  File(File parent, String child);
3  File(String pathname);
4  File(String parent, String child);
5  File(URI uri);
6
7  // 文件属性
8  boolean canRead();
9  boolean canWrite();
10 boolean exists();
11 boolean isFile();
12 boolean isDirectory();
13 long length();
14 boolean setReadOnly();
15
16
17 // 文件的创建、删除、重命名等
18 String getName();
19 String getPath();
20 String getAbsolutePath();
21 String getParent();
22 File getParentFile();
23 boolean createNewFile();
24 boolean mkdir();
25 boolean mkdirs();
26 boolean delete();
27 boolean renameTo(File f);

```

3. I/O 流

InputStream

- `FileInputStream` 用于从本地文件读取数据
- `PipedInputStream` 用于从管道读取数据
- `FilterInputStream` 的子类
 - `PushBackInputStream` 可以将读出的数据回退到缓冲区中
 - `BufferedInputStream` 可指定缓冲区的大小进行构造
 - `DataInputStream` 可以读取基本数据类型
 - `ByteArrayInputStream` 可以将指定的 `byte[]` 作为缓冲区
 - `SequencedInputStream` 可以把多个 `InputStream` 对象转化为单个
 - `ObjectInputStream` 用于读取对象
 -

```

1  int read();
2  int read(byte[] buffer);
3  int read(byte[] buffer, int offset, int length);
4
5  void close();
6  int available();
7  long skip(long n);
8  boolean markSupported();
9  void mark(int readlimit);
10 void reset();

```

OutputStream

- 基本成员与 `InputStream` 类似。
- 子类 `PrintStream` 支持多种格式的打印函数。

```

1  void write(int c);
2  void write(byte[] buffer);
3  void write(byte[] buffer, int offset, int length);
4
5  void close();
6  void flush();

```

Reader

- 结构

```

1      BufferedReader <- LineNumberReader
2      CharArrayReader
3      StringReader
4  Reader <-
5      InputStreamReader <- FileReader
6      PipedReader
7      FilterReader <- PushbackReader
8      ...

```

- 常用函数

```

1  int read();
2  int read(char[] cbuf);
3  int read(char[] cbuf, int offset, int length);
4  String readLine();
5
6  void close();
7  boolean ready();
8  long skip(long n);
9  boolean markSupported();
10 void mark(int readAheadLimit);
11 void reset();

```

Writer

• 结构

```
1      BufferedWriter
2      CharArrayWriter
3      StringWriter
4  Writer <- OutputStreamWriter <- FileWriter
5      PrintWriter
6      PipedWriter
7      FilterWriter
8      ...
```

• 常用函数

```
1  void write(int c);
2  void write(char[] cbuf);
3  void write(char[] cbuf, l int offset, int length);
4  void write(String string);
5  void write(String string, int offset, int length);
6
7  void close();
8  void flush();
```

• 过滤流

- `FilterInputStream` 和 `FilterOutputStream` 是两个**抽象类**，都属于过滤流类。
- **过滤流具有将流连接在一起的能力。**
- 过滤流实现了对象的**同步机制**，使得任一时刻只有一个线程访问它。
- 过滤流及其子类的构造通过可以接受一个 I/O 流对象作为参数，从而为之加上一层**过滤器**。

举例说明：带缓冲的字节输入流 `BufferedInputStream`

- 此类接收一个 `InputStream` 对象作为自己的成员，从而使原输入流对象带上了缓冲，并提升了读取速度。
- 此类自带缓冲区，也可以调用重载的构造指定缓冲区大小，一般以本设备的内存页或磁盘块大小的整数倍为宜。

```
1  // Example.java
2  import java.io.BufferedInputStream;
3  import java.io.File;
4  import java.io.FileInputStream;
5  import java.io.InputStream;
6  import java.util.Date;
7
8  public class Example {
9      public static void main(String[] args) {
10         File f = new File("./a.jpeg");           // 一张很大的图片
11         int c, cnt1 = 0, cnt2 = 0;
12         if (!f.exists()) System.exit(-1);        // 判断文件是否存在
```



```

13
14     Date d1 = new Date();                // 第一个时间点
15
16     try {
17         InputStream is = new FileInputStream(f);
18         while ((c = is.read()) != -1) cnt1++;    // 逐字节遍历
19         is.close();
20     } catch (Exception e) {
21         e.printStackTrace();
22         System.exit(-1);
23     }
24
25     Date d2 = new Date();                // 第二个时间点
26
27     try {
28         BufferedInputStream bis
29             = new BufferedInputStream(new FileInputStream(f));
30         while ((c = bis.read()) != -1) cnt2++;    // 逐字节遍历
31         bis.close();
32     } catch (Exception e) {
33         e.printStackTrace();
34         System.exit(-1);
35     }
36
37     Date d3 = new Date();                // 第三个时间点
38
39     System.out.println
40         ("InputStream: read " + cnt1 + " bytes, using " +
41          (d2.getTime() - d1.getTime()) + " ms.");
42     System.out.println
43         ("BufferedInputStream: read " + cnt2 + " bytes, using
44  " +
45          (d3.getTime() - d2.getTime()) + " ms.");
46 }

```

- 数据流

使用举例

```

1  import java.io.*;
2
3  public class Example {
4      public static void main(String[] args) {
5          byte[] b = {23, 90, -1, 48, -29};
6          File out = new File("./date.dat");
7          try {
8              out.createNewFile();
9
10             DataOutputStream dos
11                 = new DataOutputStream(
12                     new BufferedOutputStream(

```

```

13         new FileOutputStream(out)));
14
15     dos.write(b);                // 写入五个字节
16     dos.writeBoolean(true);      // 写入一个布尔值
17     dos.writeDouble(3.14);       // 写入一个双精度浮点数
18     dos.writeUTF("你好, 世界");  // 写入 UTF 编码的字符串
19     dos.writeChar('s');          // 写入一个字符
20     dos.writeUTF("hello 数字123"); // 写入 UTF 编码的字符串
21     dos.close();
22
23     File in = out;
24     DataInputStream dis
25         = new DataInputStream(
26             new BufferedInputStream(
27                 new FileInputStream(in)));
28
29     System.out.println("available: " + dis.available());
30     // 可读字节数
31     System.out.println("mark supported? " + dis.markSupported());
32     // 是否可标记
33     dis.skip(3);
34     // 跳过三个字节
35     System.out.println("=== skip 3 bytes, read ===\n" +
36         dis.readByte() + ", " + dis.readByte());
37     // 读取剩下两个字节的整数 48, -29
38     System.out.println(dis.readBoolean());
39     // 读取布尔类型 true
40     System.out.printf("%.2f\n", dis.readDouble());
41     // 读取双精度浮点类型 3.14
42     dis.mark(1);
43     // 在当前位置做标记
44     System.out.println(dis.readUTF()); // 读取字符串 "你好, 世界"
45     System.out.println(dis.readChar()); // 读取字符 'c'
46     System.out.println(dis.readUTF()); // 读取字符串 "hello 数字
123"
47     dis.reset(); // 回到标记位置
48     System.out.println(dis.readUTF()); // 读取字符串 "你好, 世界"
49 } catch (Exception e) {
50     e.printStackTrace();
51     System.exit(-1);
52 }
53 }
54 }

```

• 回压流

- 一些情况下需要先读入一个或几个字节，判断输入流的属性。在正式读取时，再将全部内容读出。回压字节流 `PushbackInputStream` 提供的 `unread` 函数可以把读过的一个或几个字节回压到流中，也可以回压别的字节数据。
- 构造

```

1 public PushbackInputStream(InputStream in);
2 public PushbackInputStream(InputStream in, int size); // size - 缓冲区大小

```

- **unread 函数**

```

1 public void unread(int b);
2 public void unread(byte[] b);
3 public void unread(byte[] b, int off, int len);

```

- **打印流**

- `PrintStream` 的对象**建构在其他输出流的对象的基础上**，所以其构造需要传入其他 `OutputStream` 的子类的对象作为参数。
- 标准输出 `System.out` 和标准错误 `System.err` 就是 `PrintStream` 实例的引用。
- `PrintStream` 提供了**重载**的 `write` `print` `println` 函数用于输出。
- `PrintStream` 的成员函数**一般不抛异常**，且当输出回车换行时，会**自动强制刷缓存 (flush)**。

- **管道流**

- 管道数据流一定是**成对出现、相互连接**的。管道主要用于**线程间的通信**。
- 管道输入流用管道输出流对象构造，管道输出流用管道输入流对象构造。如果连接了已经和别的管道连接的管道，会抛 `IOException`。
- 管道流是 `InputStream` 或 `OutputStream` 的子类，因而可以连接到别的流。

4. try with resource

- 打开的流越多，`try-catch-finally` 嵌套得越深。一般地，`finally` 中进行**流的关闭**，这一功能可以用 `try with resource` 语句实现。
- 凡是实现了 `java.lang.AutoCloseable` 或 `java.io.Closeable` 接口的类，都可以作为**资源 (resource)**。在 `try` 结束后，不论是否抛异常，资源都会被关闭，**关闭的顺序与资源初始化的顺序相反**。

```

1 class ResourceA implements AutoCloseable {
2     public void f() {
3         System.out.println("ResourceA");
4     }
5     @Override
6     public void close() throws Exception {
7         System.out.println("close ResourceA");
8     }
9 }
10 class ResourceB implements AutoCloseable {
11     public void f() {
12         System.out.println("ResourceB");
13     }
14     @Override
15     public void close() throws Exception {

```

```

16         System.out.println("close ResourceB");
17     }
18 }
19 public class Test {
20     public static void main(String[] args) {
21         try (ResourceA a = new ResourceA();    // 多个资源的初始化, 用分号隔开
22             ResourceB b = new ResourceB()) {
23             a.f();
24             b.f();
25         } catch (Exception e) {
26             e.printStackTrace();
27         }
28     }
29 }

```

5. 对象流和串行化

- 使用对象流 `ObjectInputStream` 和 `ObjectOutputStream` 将从文件中读取某个类的对象或把某个类的对象存到文件中。
- 能够进行串行化的类必须实现串行化接口 `java.io.Serializable`。此接口中没有定义任何内容, 仅用于告诉编译器这个类可以串行化。串行化 (Serialize) 指的是把自身转化为一系列字节以记录状态。
- 有些类不能进行串行化, 如 `Thread` 对象、`FileInputStream` 对象等, 因为其状态是暂时的。对不希望串行化的对象应使用 `transient` 关键字修饰。

```

1  import java.io.*;
2
3  class MyClass implements Serializable {
4      int i;
5      double d;
6      transient String s1;    // 不被串行化的成员
7      boolean b;
8      String s2;
9      MyClass(int i, double d, String s1, boolean b, String s2) {
10         this.i = i; this.d = d; this.s1 = s1; this.b = b; this.s2 = s2;
11     }
12     @Override
13     public String toString() {
14         return new String("i = " + i + ", d = " + d +
15             ", s1 = " + s1 + ", b = " +
16             b + ", s2 = " + s2);
17     }
18 }
19
20 public class Test {
21     public static void main(String[] args) {
22         MyClass mc = new MyClass(10, 3.14, "hello", true, "world");
23         File f = new File("./class.dat");
24         try {
25             f.createNewFile();
26

```

```
27         ObjectOutputStream oos
28             = new ObjectOutputStream(
29                 new FileOutputStream(f)); // 创建对象输出流
30         oos.writeObject(mc); // 将对象 mc 写入文件
31         oos.close();
32
33         ObjectInputStream ois
34             = new ObjectInputStream(
35                 new FileInputStream(f)); // 创建对象输入流
36         MyClass mc1 = (MyClass)ois.readObject();
37                                     // 从文件读取对象(readObject 返回
Object)
38         ois.close();
39
40         System.out.println(mc1); // s1 未被序列化, 故会输出默认值 s1 = null
41     } catch (Exception e) {
42         e.printStackTrace();
43     }
44 }
45 }
```